Presentable Debugging with LLVManim

Benjamin Houle
Dept. of Computer Science
University of Massachusetts Lowell
Lowell, Massachusetts, USA
Benjamin_Houle@student.uml.edu

Ebenezer Oloyede
Dept. of Computer Science
University of Massachusetts Lowell
Lowell, Massachusetts, USA
Ebenezer_Oloyede@student.uml.edu

Lily Grippo
Dept. of Computer Science
University of Massachusetts Lowell
Lowell, Massachusetts, USA
Lily_Grippo@student.uml.edu

Mohammad Ahmed
Dept. of Computer Science
University of Massachusetts Lowell
Lowell, Massachusetts, USA
ahmedzubair16dec@gmail.com

***Abstract*—LLVManim is a code visualization tool that generates aesthetically pleasing animations of control flow and memory operations from LLVM IR. Over the course of a single semester, we designed and implemented a three-layer pipeline—Ingestion, Transformation, and Rendering—that parses `.ll` files with `llvmlite`, derives a typed intermediate scene graph, and renders animations through Manim Community Edition. The finished tool supports three animation modes (basic stack view, rich IR spotlight, and a three-column SSA panel), a CFG traversal animation driven by Graphviz layout, and machine-readable JSON and DOT export. The implementation comprises 26 Python modules validated by 302 automated tests at 84% line coverage, enforced by a four-stage quality gate (Ruff, Pyright, import-linter, pytest). LLVManim demonstrates that presentation-quality program visualization can be made accessible through a simple command-line interface, eliminating the need for manual Manim scripting or post-processing of LLVM's utilitarian graph output.**

CCS Concepts: **• Human-centered computing → Visualization → Visualization systems and tools → Visualization toolkits; • Software and its engineering → Software notations and tools → Compilers; • Applied computing → Education → Interactive learning environments**

***Index Terms*—program visualization, code animation, LLVM, static analysis, control flow graphs, memory visualization, educational tools, Manim, compiler infrastructure, presentation tools**

I. Introduction

Despite static analysis and data visualization being among the oldest concepts in computer science, disappointingly few graphic designers have been significantly interested in the former. While plenty of tools exist for creating control flow graphs or visualizing the stack, the majority of these favor a "function over form" paradigm that focuses on creating lightweight, readable output, assuming the user is more concerned with the information than its presentation. This, however, is not always the case: presenting in any context, such as at a conference or school, generally favors quickly digestible and visually appealing content over static, large blocks of text. In these cases, the presenter is forced to choose between using the unappealing generated visualization and the slog of creating something presentable for themselves.

LLVManim was developed to address this problem. By pairing LLVM's powerful static analysis infrastructure with the Manim animation library, the tool automatically generates professional-quality visualizations from LLVM IR and runtime traces. Through a simple command-line interface, users can produce animated stack walkthroughs, IR spotlight views with SSA value tracking, CFG traversal animations, and machine-readable scene graph exports—all without writing any Manim code or manually post-processing compiler output.

Over the course of a single semester, we designed and built a three-layer pipeline (Ingestion, Transformation, and Rendering) comprising 26 Python modules and validated by 302 automated tests at 84% line coverage. The remainder of this paper describes the tools and techniques that informed our design (Sections II–III), the system architecture (Section IV), the results of our implementation (Section V), lessons learned (Section VI), and conclusions (Section VII).

II. Related Work

A. PythonTutor

PythonTutor [1] is a widely used educational tool for visualizing program execution, originally developed to help novices build accurate mental models of how code runs. Its core contribution is a step-by-step execution trace—illustrated in Fig. 1—that exposes stack frames, heap objects, and control flow in a concrete, visual manner. As described by its creator, Philip J. Guo, PythonTutor aims to "help people overcome a fundamental barrier to learning programming: understanding what happens as the computer runs each line of code" [2].

PythonTutor has been shown to be effective in improving students' understanding of programming concepts; Guo's further studies demonstrated significant learning gains when PythonTutor was used as a supplement to traditional instruction [3]. This demonstrates the educational value of connecting code execution to visual representations, as well as the benefits of an accessible, interactive tool.

However, the visualizations produced by PythonTutor are relatively basic and not suitable for presentation purposes. PythonTutor's primary focus is on clarity and educational

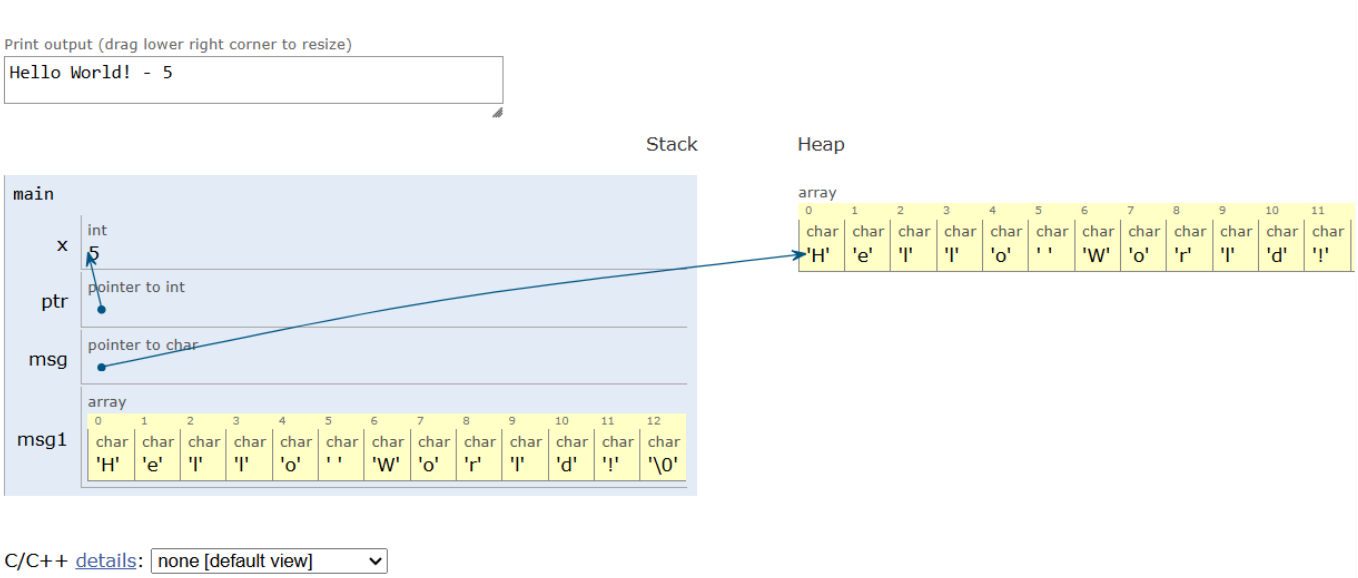


Fig. 1: Example PythonTutor.com output showing a step-by-step execution trace with stack frames and heap objects.

value rather than aesthetics, resulting in output that may be difficult to read or visually unappealing when projected to an audience. This, along with the fact that the tool is web-based and not designed for offline use, makes it less than ideal for use in settings such as conference presentations where visual impact is important. PythonTutor is additionally restricted by its limited customization options, minimal library of visualization methods, and support for only C, C++, Java, Javascript, and Python.

B. LLVM Analysis and Transform Passes

LLVM has emerged as the de-facto standard compiler infrastructure for modern software development, powering everything from the development environment for Apple operating systems to cutting-edge systems research efforts. Its adoption by the compiler community can be attributed to several key strengths—a modular architecture that separates language frontends from optimization backends, an extensive suite of analysis and transformation passes that expose deep insights into program behavior, and robust support for over 30 programming languages, including those supported by PythonTutor. LLVM even provides users with an API to define their own language [4].

At the core of LLVM's value proposition for code analysis is its Intermediate Representation (IR), a low-level and platform-independent format that captures program semantics in a standardized way. This IR serves as the foundation for powerful built-in analysis passes that can generate control flow graphs, dominator trees, data flow analyses, and detailed memory access patterns. For compiler developers and researchers, these capabilities are invaluable—LLVM effectively solves the hard problem of extracting structured information from code, allowing them to focus on higher-level tasks like optimization or verification [5].

However, LLVM's visualization capabilities tell a different story. The standard approach for visualizing analysis results relies on generating DOT[1] files through passes like `-dot-cfg` or `-view-cfg`, which are then rendered using Graphviz [6]. While functionally correct, the resulting diagrams—an example of which is shown in Fig. 2—embody a strictly utilitarian aesthetic: monochrome nodes and edges, densely packed labels filled with LLVM IR instructions, and layouts optimized for completeness rather than clarity. This "function over form" philosophy serves well enough as a developer tool for debugging optimizer passes, but it fails as a solution for communication beyond that narrow domain.

The barrier extends beyond mere aesthetics. Interpreting these diagrams requires familiarity with LLVM IR syntax—understanding what "`%4 = load i32, i32* %3`" means demands compiler knowledge that most audiences lack. The visualizations are fundamentally static images with no mechanism for progressive disclosure or animation, forcing presenters to mentally simulate execution flow while pointing at a frozen graph. Moreover, customization options are severely limited; changing colors, adjusting layouts, or highlighting specific execution paths requires manual post-processing in external tools, completely defeating the purpose of automated generation.

In essence, LLVM provides extraordinarily powerful analysis capabilities but pairs them with visualization tools that assume their primary consumer is someone who already

[1] Abstract grammar for defining Graphviz nodes, edges, graphs, subgraphs, and clusters.

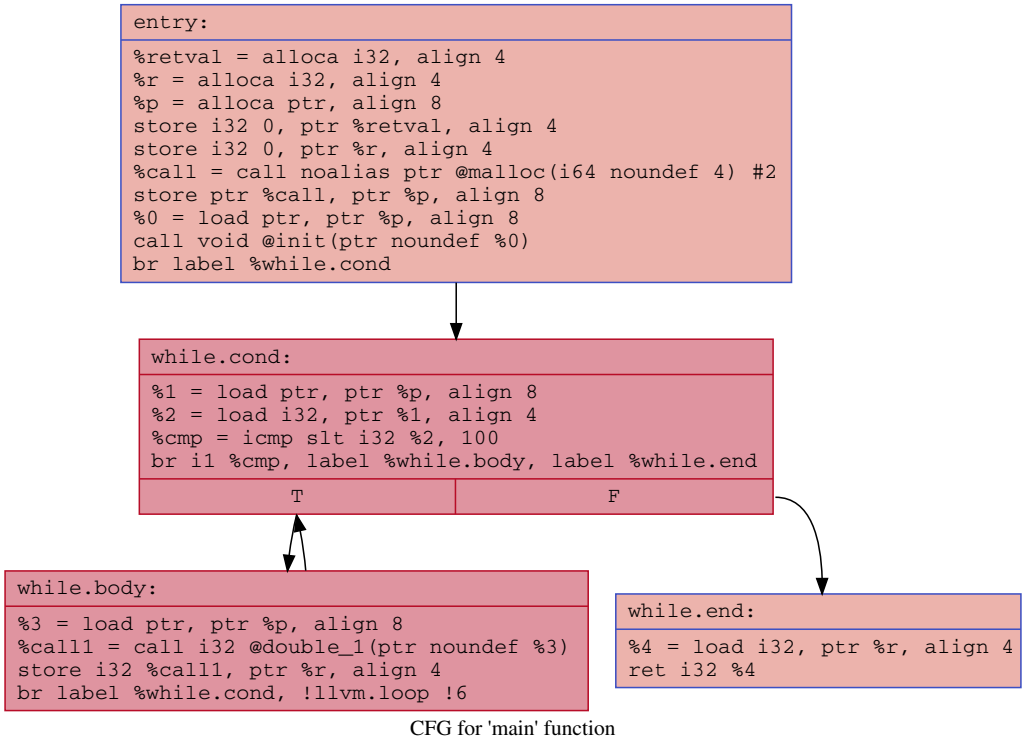


Fig. 2: Example Control-Flow Graph (CFG) using LLVM `-dot-cfg` pass. While technically accurate, the visualization prioritizes information density over visual clarity.

understands what they are looking at. This creates a frustrating gap: rich semantic information about program behavior can be automatically extracted, but it cannot be automatically presented in a way that serves educational or presentational contexts. LLVManim was built to bridge this gap by maintaining LLVM's analytical rigor while replacing its austere output with animations designed for human communication rather than machine precision.

III. Manim

The Manim library [7] was originally developed by Grant Sanderson [8] for his educational YouTube channel "3Blue1Brown" [9] to create animations of mathematical and computer-science concepts. Since its release, the channel has accumulated over 6 million subscribers, demonstrating the educational impact of smooth, precisely choreographed animations (Fig. 3) that build intuition about abstract concepts. Given this history, Manim represented an excellent candidate for creating high-quality educational visualizations of program behavior.

Manim's strength lies in its ability to programmatically generate animations with mathematical precision. Instead of tedious keyframe animation, users write Python code to determine how objects should appear, change, and interact. Manim can render 3D scenes, typeset LaTeX equations, and output crisp 60 fps animations suitable for anything from YouTube tutorials to academic conferences. For education, its capacity to make complicated ideas—like matrix transformations, function composition, or program flow—tangible and understandable by systematically unfolding them is particularly valuable.

Yet this power comes with a barrier to entry. A professor preparing a single lecture, or a researcher creating visualizations for a conference talk, may have a clear idea of what they want to show—perhaps how a recursive function builds up its stack—but writing Manim code for it requires a distinct skill set. LLVManim addressed this challenge by automatically translating program semantics into Manim scenes, thereby eliminating the barrier of manual coding between LLVM's analytical power and Manim's visual sophistication.

IV. LLVManim System Design

LLVManim bridges the gap between two observations:

1) Methods exist to quickly generate code visualizations, but they produce output that is unappealing for presentation purposes.
2) The Manim library can produce visually impressive animations, but using it requires significant manual effort.

By automating the translation from LLVM IR to Manim scenes, educators and tech leads alike can produce presentation-ready visualizations without needing to compromise between style and accessibility.

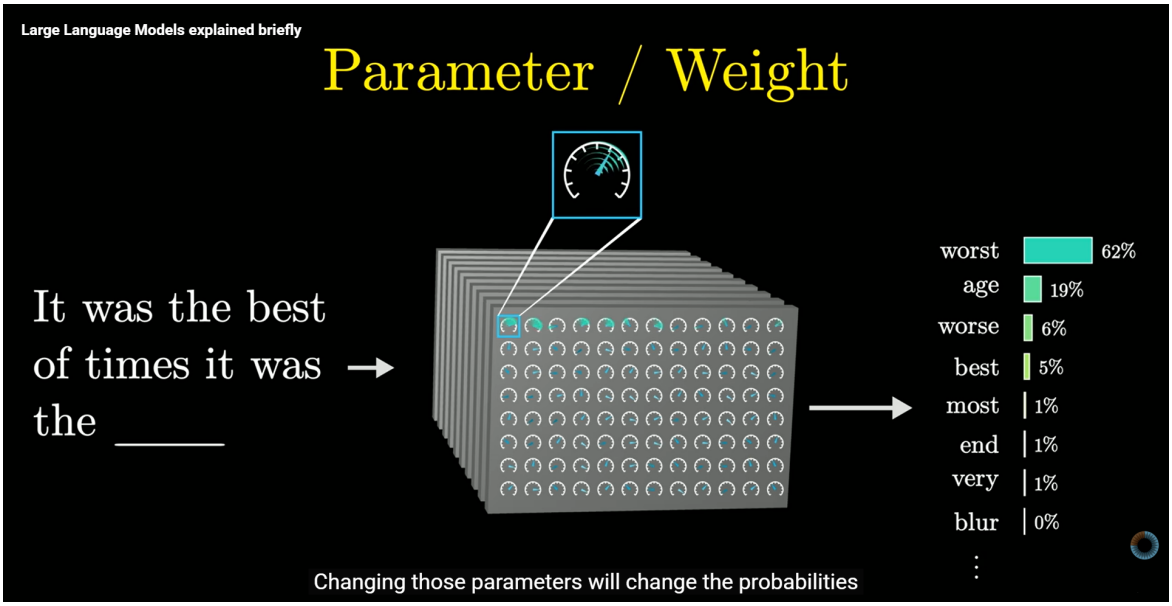


Fig. 3: An example of a Manim animation from the 3Blue1Brown YouTube video: Large Language Models explained briefly [10].

A. Design Goals

The following design requirements guided the implementation of LLVManim:

- **Simple CLI:** A command-line interface that accepts `.ll` files, selects visualization options via flags, and produces output with minimal configuration.
- **LLVM IR Parsing:** Extraction of control flow, memory operations, and SSA value information from LLVM IR using the `llvmlite` [11] Python binding.
- **Multiple Animation Modes:** Several rendering modes offering different levels of detail—from a compact stack-only view to a full three-column layout showing IR source, SSA values, and the call stack simultaneously.
- **IR Instruction Coverage:** Support for the most common LLVM IR instructions (`alloca`, `load`, `store`, `call`, `ret`, `br`, binary operations, and comparisons), enabling visualization of a wide range of programs.
- **Machine-Readable Export:** JSON scene graph and Graphviz DOT output for integration with external tools and downstream analysis.
- **Testability and Quality:** A comprehensive automated test suite and multi-stage quality gate to maintain correctness throughout development.

Deliverables:

- A command-line tool that accepts LLVM IR as input and produces animations, DOT graphs, or JSON scene data as output.
- A library of reusable Manim scene components for visualizing control flow and stack operations.
- Sample visualizations demonstrating LLVManim on standard test programs.
- Documentation covering installation, usage, CLI flags, and supported IR constructs.
- A test suite of 302 automated tests with 84% line coverage.

B. Architecture

LLVManim is implemented as a three-layer pipeline—**Ingestion**, **Transformation**, and **Render**—orchestrated by a dedicated **CLI** entry point. The high-level structure is shown in Fig. 4. Layer boundaries are enforced at build time by `import-linter`, ensuring that no module imports from a downstream layer. The entire codebase is written in Python 3.12 and managed with `uv`.

1) CLI Entry Point: The `llvmanim.cli` module is the user-facing entry point. It accepts LLVM IR files and orchestrates the three pipeline layers via `argparse`:

```
# Export JSON scene graph + Graphviz DOT/PNG
$ uv run llvmanim input.ll --json --draw \
    --outdir out/
# Render basic stack animation
$ uv run llvmanim input.ll --animate \
    --ir-mode basic --outdir out/
# Render rich IR spotlight with SSA panel
$ uv run llvmanim input.ll --animate \
    --ir-mode rich-ssa --speed 1.5 --outdir out/
# CFG traversal animation
$ uv run llvmanim input.ll --cfg-animate \
    --dot-cfg main.dot --import-trace trace.json \
    --outdir out/
```

Listing 1: Actual CLI usage

Fig. 5 traces the complete CLI dispatch flow, showing how each command flag routes through the shared Ingest pipeline to the appropriate Transform and Render stages. When the tool encounters unsupported IR constructs, missing debug metadata, or malformed input, it emits a descriptive error message and continues processing the remainder of the program where possible.

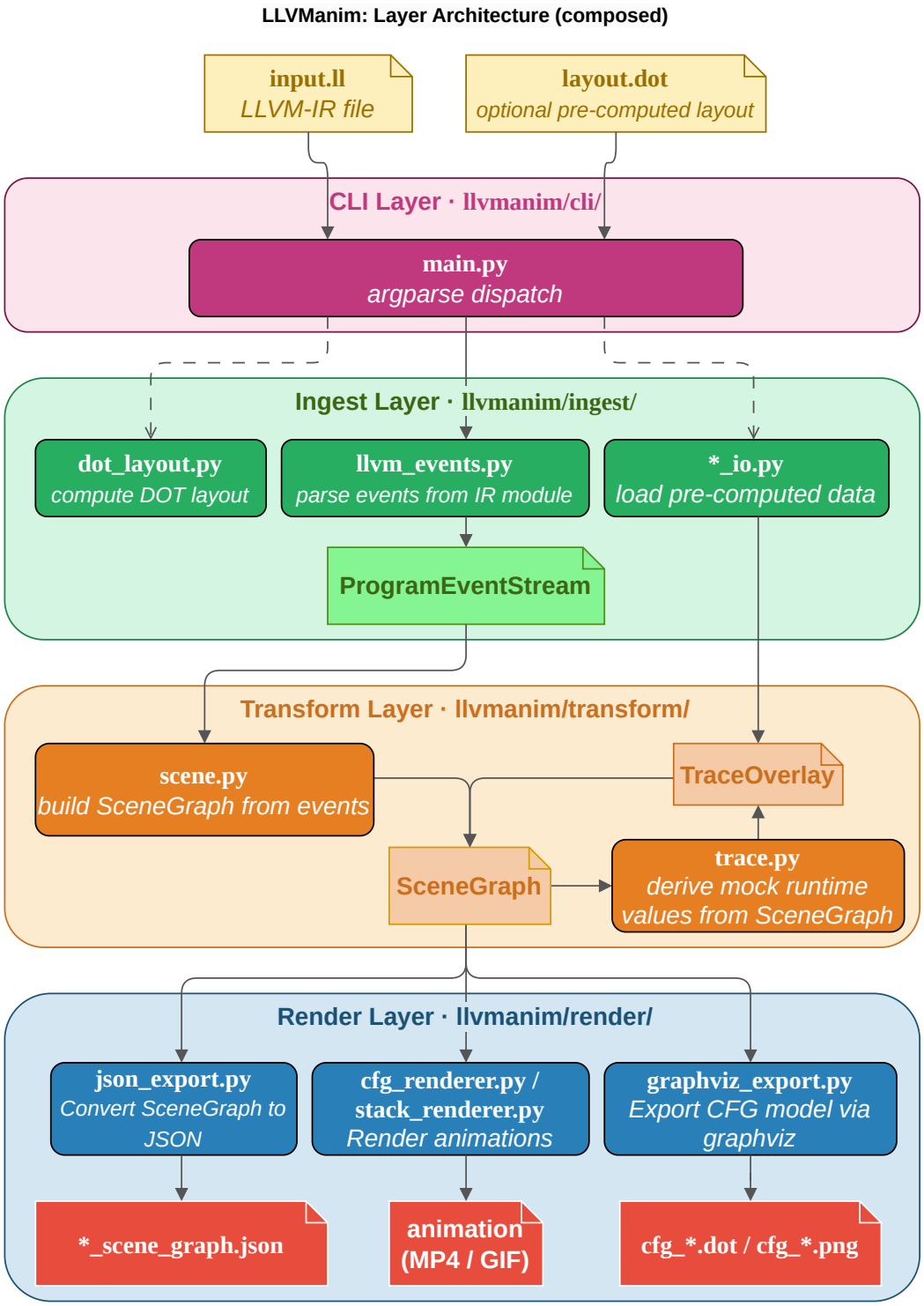


Fig. 4: Detailed module-level architecture. The Ingestion layer collects LLVM IR and metadata, the Transformation layer maps it to an intermediate scene graph, and the Rendering layer renders animations or exports artifacts.

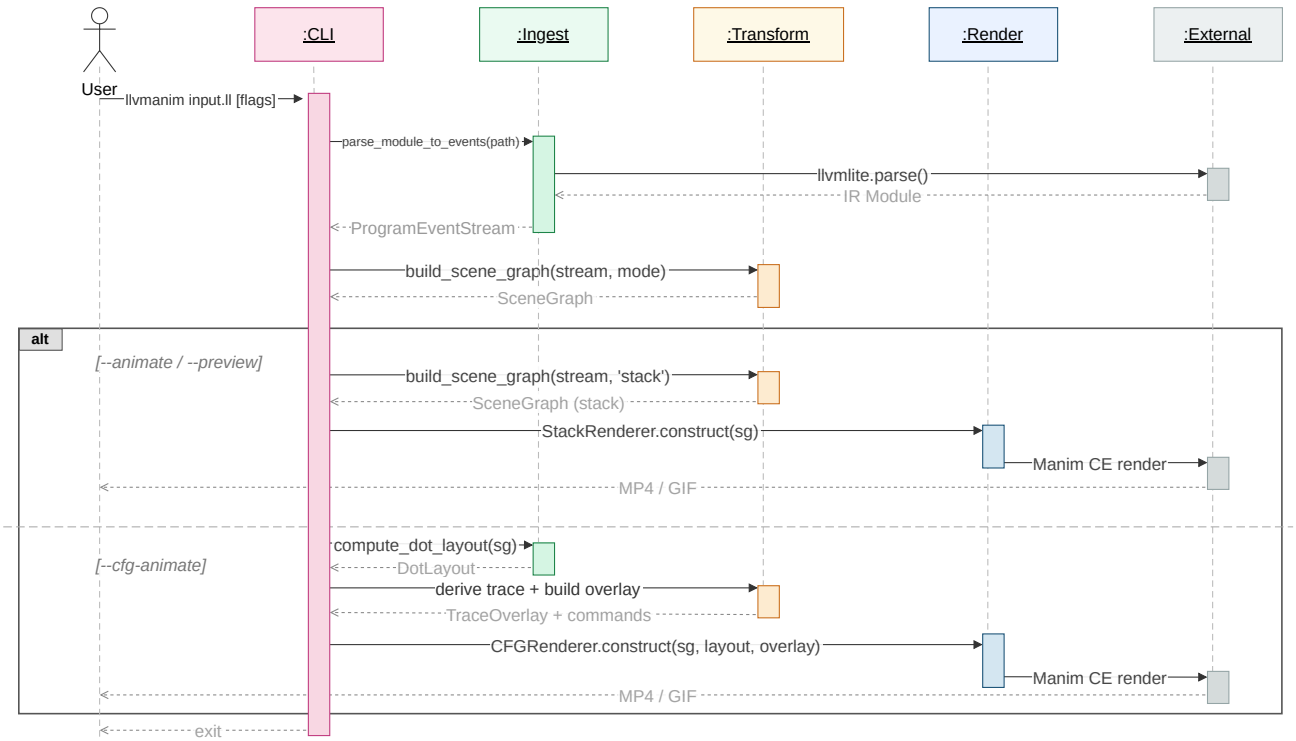


Fig. 5: CLI dispatch flow. Each command flag (`-json`, `-draw`, `-animate`, `-cfg-animate`) shares the Ingest pipeline and branches to the appropriate Transform and Render stages.

2) Ingestion Layer: The Ingestion layer extracts program structure from LLVM IR. It uses the `llvmlite` [11] Python binding to parse `.ll` files and also supports importing pre-computed data from JSON.

- **`llvm_events.py`:** Parses an LLVM module and emits a `ProgramEventStream`—a typed sequence of `IREvent` records. Each event carries the function name, block name, opcode, full instruction text, a classified `EventKind` (one of `alloca`, `load`, `store`, `binop`, `compare`, `call`, `ret`, `br`, or `other`), its index within the function, debug line number, and operand list. Typed CFG edges are extracted directly from `llvmlite` terminator operands (`br`, `switch`, `invoke`, `indirectbr`, `callbr`) with true/false labels on conditional branches.
- **`cfg_edge_io.py`:** Serializes and deserializes `CFGEdge` lists to JSON, enabling round-trip import/export of control-flow edges via dedicated CLI flags.
- **`analysis_metadata_io.py`:** Loads and saves dominator-tree and loop-analysis metadata as JSON, supporting round-trip import and export via CLI flags.
- **`trace_io.py`:** Reads and writes runtime path traces as JSON (`-import-trace` / `-export-trace`), used for overlay visualization in CFG traversal animations.
- **`dot_layout.py`:** Parses a Graphviz DOT file produced by `opt -passes=dot-cfg` and computes a `DotLayout` with node positions and edge routes for the CFG animation scene.
- **`display_lines.py`:** Cleans raw IR text into per-function line lists used by the rich animation panels.

3) Transformation Layer: The Transformation layer converts the ingest output into an intermediate scene graph and execution traces suitable for rendering. The complete data model spanning all three layers is shown in Fig. 6.

- **`models.py`:** Defines the core data model—`SceneGraph`, `SceneNode`, `SceneEdge`, `CFGBlock`, `TraceOverlay`, and `BlockMetadata`. Also defines `AnimationCommand` and `ActionKind`, the typed animation primitives consumed by the Render layer. Nodes carry a `kind` string and an open `properties` dict, with an optional `animation_hint` string. `SceneGraph` embeds an ordered `commands` list alongside its nodes and edges.
- **`scene.py`:** The `build_scene_graph` function groups events by function and block, attaches typed CFG edges, assigns block roles, applies optional domtree/loop metadata, and populates `SceneGraph.commands` via the overlay builder. `build_stack_scene_graph` produces a call-tree-ordered `SceneGraph` with a fully populated `commands` list, translating the parsed `ProgramEventStream` directly into typed `AnimationCommand` records (carrying `ActionKind`,

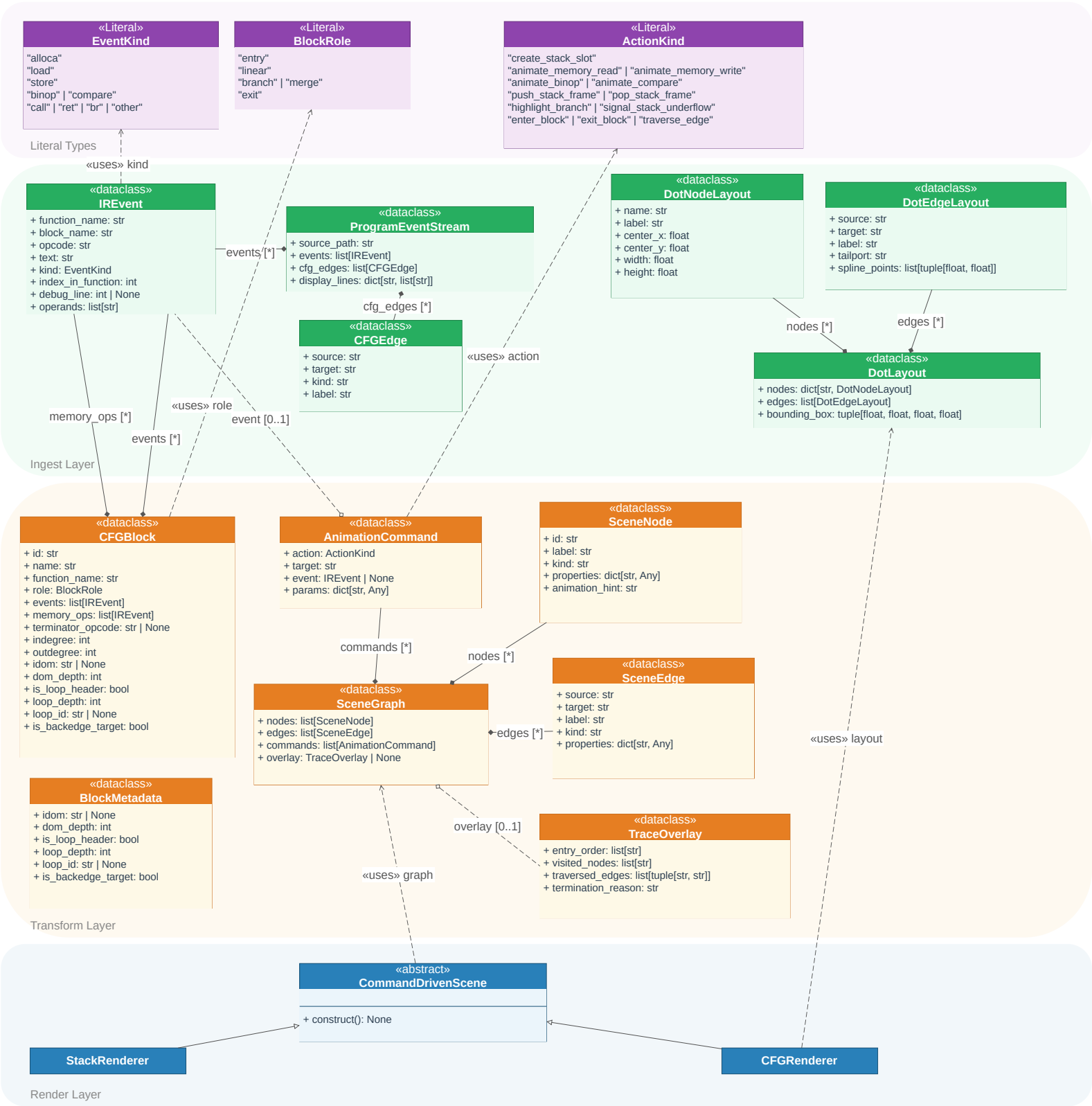


Fig. 6: Unified data model. Ingest types (`ProgramEventStream`, `IREvent`, `CFGEdge`, `DotLayout`) feed into Transform types (`SceneGraph`, `SceneNode`, `SceneEdge`, `AnimationCommand`, `TraceOverlay`) which are consumed by Render types (`CommandDrivenScene`, `StackRenderer`, `CFGRenderer`).

target, and params) ready for consumption by the Render layer.

- **trace.py:** The build_execution_trace function produces a list of TraceStep or RichTraceStep records capturing the execution order with optional SSA result values for binop, compare, and load instructions.

4) Render Layer: The Render layer renders the scene graph using Manim Community Edition [12] and provides export functionality. All scenes extend CommandDrivenScene, a base class that iterates over the SceneGraph.commands list and dispatches each AnimationCommand to a registered handler. The complete type hierarchy—from Ingest types through Transform intermediates to the Render dispatch classes—is shown in Fig. 6.

- **Animation Modes:** The tool supports three stack-rendering modes, all driven by StackRenderer—a CommandDrivenScene subclass. Each mode consumes the same AnimationCommand records (detailed in Fig. 6) but presents progressively more detail.
  - -ir-mode basic: Stack-only view with per-cell badge flash for each instruction.
  - -ir-mode rich: Two-column layout with an IR source panel and a moving spotlight cursor alongside the stack view.
  - -ir-mode rich-ssa: Three-column layout (IR Source | SSA Values | Stack) showing computed results for binary operations, comparisons, and loads.
- **CFG Traversal:** -cfg-animate renders an animated walk through the control-flow graph using CFGRenderer (another CommandDrivenScene subclass) with DOT-derived node positions (-dot-cfg) and a runtime trace (-import-trace). If no trace is provided, a static DFS trace is derived automatically.
- **Export:** Static artifacts complement the animated output.
  - -json: Writes a machine-readable scene graph (scene_graph.json).
  - -draw: Writes cfg_main.dot and, when Graphviz is available, a rendered cfg_main.png.
- **Rendering:** Manim CE renders MP4 or GIF output. GIF conversion uses an ffmpeg palette workflow to reduce peak memory usage. The -preview flag opens the rendered animation in a viewer.

V. Results

A. Feature Coverage

Table I summarizes the proposed features from the original project goals alongside their implementation status.

Of the originally proposed proof-of-concept targets, all were achieved except automatic C-to-IR compilation (users supply .ll files directly) and interactive stepping. The three-column SSA panel and CFG traversal animation were implemented as additional capabilities beyond the original scope.

TABLE I: Proposed vs. Implemented Features

Feature	Status
CLI accepting .ll files	✓
LLVM IR parsing via llvmlite	✓
Typed CFG edge extraction	✓
Intermediate scene graph	✓
JSON scene graph export	✓
Graphviz DOT/PNG export	✓
Basic stack animation	✓
Rich IR spotlight animation	✓
Rich-SSA 3-column animation	✓
CFG traversal animation	✓
Runtime trace import/export	✓
Domtree/loop metadata import	✓
GIF output with palette workflow	✓
Adjustable animation speed	✓
Automatic C compilation	×
Interactive stepping/pause	×
YAML configuration files	×
Color palette selection	×
GUI	×

B. Supported IR Instructions

LLVManim classifies each LLVM IR instruction into one of nine EventKind categories:

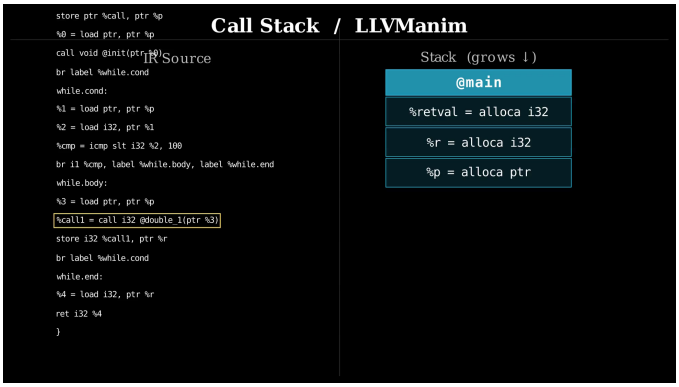
- alloca: Stack memory allocation
- load / store: Memory read and write operations
- binop: Binary arithmetic operations (add, sub, mul, etc.)
- compare: Integer and floating-point comparisons (icmp, fcmp)
- call: Function invocations
- ret: Function returns
- br: Branch instructions (conditional and unconditional)
- other: Unsupported or unrecognized instructions (skipped with a diagnostic message)

CFG edge extraction supports five terminator types: br, switch, invoke, indirectbr, and callbr. Conditional branches are labeled with T/F direction annotations.

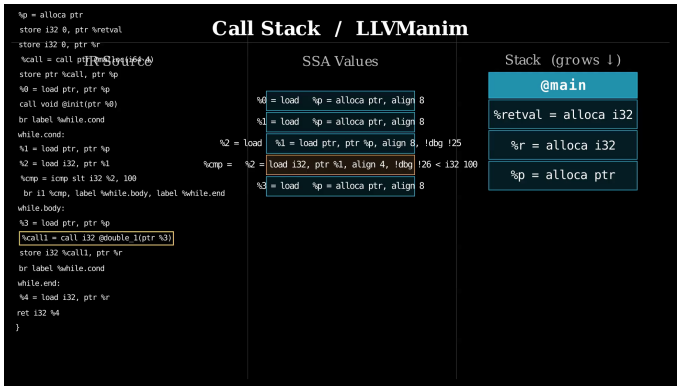
C. Animation Modes

LLVManim provides three stack-focused animation modes and one graph-based mode, each reproducible with a single CLI command. Fig. 7 shows representative frames from all three stack modes. The LLVM IR input file (double.ll) used to produce all sample outputs is listed in Appendix B.

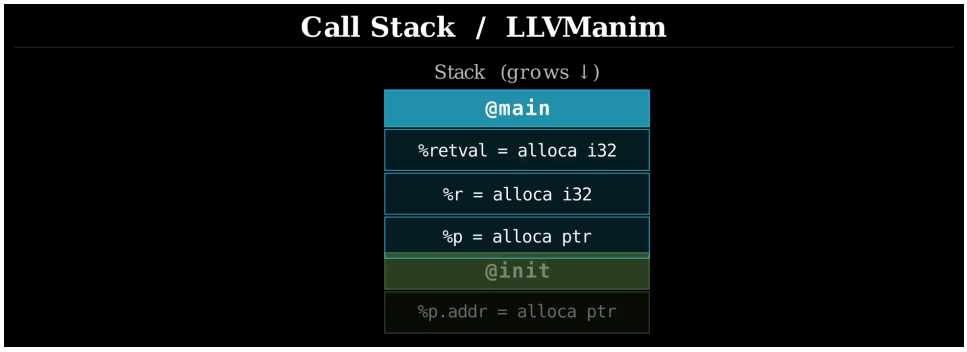
Basic mode (-ir-mode basic) renders a minimal stack-only view. Each IR instruction triggers a per-cell badge flash on the corresponding stack slot, providing a clean visual pulse of execution without additional clutter. This mode is best suited for audiences who need only a high-level sense of execution flow, such as introductory lectures or quick demonstrations, since it strips away all source-level detail and focuses purely on the call stack.



(a) Rich mode: two-column layout with IR source spotlight alongside the stack.



(b) Rich-SSA mode: three-column layout showing computed SSA values for `load` and `icmp` instructions.



(c) Basic mode: stack-only view with per-cell badge flash during a `call` to `@init`.

Fig. 7: The three stack-focused animation modes, shown with cropped frames from actual LLVManim output.

```
uv run llvmanim double.ll --animate \
    --ir-mode basic --outdir out/
```

Rich mode (`-ir-mode rich`) extends the basic view with a two-column layout. An IR source panel on the left displays the current function's instructions while a spotlight cursor tracks the executing line, giving the audience both the "what" (the stack) and the "where" (the source). This mode is particularly valuable when the presenter wants to walk through specific IR constructs—for example, showing how a series of `alloca` and `store` instructions initialize local variables before a loop body executes. The spotlight cursor provides a natural focal point that guides the viewer's attention through the code.

```
uv run llvmanim double.ll --animate \
    --ir-mode rich --outdir out/
```

Rich-SSA mode (`-ir-mode rich-ssa`) takes the rich layout a step further with a dedicated third column for SSA values (IR Source | SSA Values | Stack). The center panel displays the symbolic result of each binary operation, comparison, or load as it executes, making it possible to follow data dependencies without mentally resolving `%`-names. This is the most detailed mode and targets audiences already familiar with SSA form—for instance, a compiler course where students need to see how `%4 = icmp slt i32 %2, %3` evaluates during a loop iteration.

```
uv run llvmanim double.ll --animate \
    --ir-mode rich-ssa --outdir out/
```

CFG traversal (`-cfg-animate`) renders an animated walk through the control-flow graph, using Graphviz DOT layout for node positions and a runtime trace for the traversal order:

```
uv run llvmanim double.ll --cfg-animate \
    --dot-cfg main.dot \
    --import-trace trace.json --outdir out/
```

D. Test Coverage and Quality

The LLVManim implementation is validated by 302 automated tests organized into four marker categories:

TABLE II: Test Distribution by Category

Category	Count
Unit (ingest, transform)	192
Integration (present, cli, pipeline)	96
Contract (export boundaries)	8
End-to-end (entrypoints)	6
Total	**302**

Line coverage stands at 84% across the entire `llvmanim` package. Fig. 8a (Appendix C) shows per-module coverage and Fig. 8b shows the test distribution across modules. A four-stage quality gate runs on every pull request:

1) **Ruff**: Linting and import sorting
2) **Pyright**: Static type checking (strict mode)
3) **import-linter**: Architectural boundary enforcement between layers
4) **pytest**: Full test suite execution with coverage reporting

E. Codebase Size

The LLVManim codebase comprises 3,189 lines of Python across 18 modules. Fig. 9 (Appendix C) shows the breakdown by layer and module. The Render layer accounts for 42% of the code, followed by Ingest (25%), Transform (19%), and CLI (14%).

F. Performance

All pipeline stages exhibit linear scaling behavior, confirming that LLVManim can handle realistically sized LLVM IR inputs. Benchmarks use synthetic IR programs with controlled instruction and block counts (median of 5 runs per data point).

Fig. 10 (Appendix C) shows the relative wall-clock cost of each pipeline stage for a 50-block CFG. The Ingest stage (`parse_module_to_events`) dominates at 11.6 ms, while all Transform stages complete in under 0.25 ms. Fig. 11 confirms that parse time scales linearly with instruction count—$\Theta(n)$ where n is the number of IR instructions.

The scene-graph construction stages also scale linearly: `build_scene_graph(mode='cfg')` is $\Theta(n_b)$ in block count (Fig. 12), `derive_cfg_trace` is $\Theta(n_b)$ for linear-chain CFGs and $O(n_b \cdot k)$ in the presence of loops (Fig. 13), and `build_scene_graph(mode='stack')` exhibits sub-linear growth empirically (Fig. 14).

Peak memory allocation also grows linearly with IR size for both pipelines (Fig. 15).

G. Reproducibility

All LLVManim results can be reproduced from the public repository [13]:

```
git clone https://github.com/BenHoule/LLVManim.git
cd LLVManim
uv sync --dev
./scripts/quality-check.sh  # runs all 4 gates
```

System requirements: Python 3.12+, `uv`, Linux packages `libcairo2-dev` and `libpango1.0-dev` (for Manim), and optionally `graphviz` for PNG graph export.

VI. Discussion and Lessons Learned

A. Architectural Decisions

Several architectural decisions shaped the final system:

Three-layer separation with import-linter. Enforcing strict layer boundaries at build time prevented coupling between Ingestion, Transformation, and Rendering. This paid dividends during the IR display model refactor, where the Transformation layer's data structures were restructured without touching any Rendering layer code. The `import-linter` tool caught accidental cross-layer imports on multiple occasions during development.

Display model abstraction. Rather than having Manim scenes query the scene graph directly, we introduced an explicit display model (the `TraceStep`/`RichTraceStep` and `AnimationCommand` types) that decouples rendering from data extraction. This made it straightforward to add the rich-SSA animation mode by extending the trace step type without modifying existing scene classes.

Pipeline approach. The tool processes data in a linear pipeline: parse $\rightarrow$ build scene graph $\rightarrow$ render or export. This simplified testing because each stage could be exercised independently with deterministic fixtures. The 302 tests in the suite are predominantly unit tests that operate on a single layer.

llvmlite for terminator parsing. Early prototypes used regex-based parsing of LLVM IR text, which proved fragile for edge cases such as `switch` and `invoke` terminators. Switching to `llvmlite`'s Python binding, which exposes the LLVM C API, made CFG edge extraction reliable across all five terminator types.

B. Deviations from Proposal

The following proposed features were not implemented:

- **Automatic C compilation:** The original proposal envisioned accepting `.c` files and invoking `clang` automatically. In practice, requiring users to supply pre-compiled `.ll` files simplified the tool's scope and avoided platform-specific compiler configuration issues.
- **Interactive stepping and pause:** Interactive playback was deprioritized in favor of richer static animation modes. Manim CE's rendering model produces video files rather than interactive sessions, making pause/step less natural.
- **YAML configuration and –palette:** Fine-grained customization via YAML files and named color palettes was deferred. The current CLI provides `-speed` and `-format` as the primary customization points.
- **GUI:** A graphical interface was listed as a stretch goal and was not pursued given the limited timeline.

Conversely, two capabilities were implemented that were *not* in the original proposal:

- **Rich-SSA three-column animation:** The `-ir-mode rich-ssa` mode displays symbolic SSA values alongside IR source and stack state, providing deeper instruction-level insight than originally planned.
- **CFG traversal animation:** The `-cfg-animate` mode renders an animated walk through the control-flow graph using Graphviz DOT layout, a feature that emerged from explorations in the sandbox environment.

C. Known Limitations

The current implementation has several known limitations:

- **Double-pop bug:** Under certain control-flow patterns involving consecutive returns, the basic animation mode may attempt to pop the stack twice for a single `ret` instruction. This is tracked as a known issue.
- **Symbolic-only SSA values:** The rich-SSA panel displays symbolic operand names (e.g., `%3 = add %1, %2`) rather than concrete runtime values. A single swap-point in `ssa_formatting.py` exists for when numeric runtime values are plumbed through in a future version.
- **No heap visualization:** The tool visualizes stack operations but does not yet render heap allocations or pointer-based data structures.
- **Manim render times:** Full-quality Manim renders can take significant time for programs with many instructions. The `-speed` flag and low-quality preview mode help during development, but render time remains a practical constraint for large programs.

D. Lessons Learned

- **TDD was essential for refactors.** The test suite caught regressions immediately when refactoring the IR display model and restructuring the scene graph. Without comprehensive tests, these refactors would have been significantly riskier.
- **Architectural enforcement catches violations early.** `import-linter` rejected several pull requests that would have introduced cross-layer dependencies, preventing technical debt before it accumulated.
- **Sandbox-driven exploration.** Experimental Manim scripts in the `sandbox/` directory allowed rapid prototyping of animation ideas (CFG traversal, register panels) without risking the main codebase.
- **Typed CFG edges via llvmlite.** Moving from regex-based IR text parsing to `llvmlite`'s terminator API eliminated an entire class of parsing bugs and made the Ingestion layer robust against IR formatting variations.

VII. Conclusion

We presented LLVManim, a tool that automatically generates presentation-quality animations from LLVM IR. The system's three-layer pipeline—Ingestion, Transformation, and Rendering—cleanly separates concerns and is enforced by architectural boundary checks at build time. Over a single semester, we delivered a fully functional CLI tool with three animation modes, a CFG traversal visualizer, machine-readable export formats, and a comprehensive test suite of 302 tests at 84% line coverage.

LLVManim demonstrates that the gap between LLVM's powerful but utilitarian analysis output and the polished animations achievable with Manim can be bridged through thoughtful automation. By targeting LLVM IR as input, the tool inherits support for any language that compiles through the LLVM toolchain, making it broadly applicable to educational and presentational contexts.

A. Future Work

Several directions remain for future development:

- **Numeric SSA values:** Plumbing concrete runtime values through the trace overlay to replace symbolic operand names in the rich-SSA panel.
- **Heap visualization:** Extending the animation modes to render heap allocations, pointer dereferences, and linked data structures.
- **Double-pop fix:** Resolving the known bug in consecutive-return control-flow patterns.
- **Automatic compilation:** Optionally invoking `clang` to compile C/C++ source files to `.ll` before processing.
- **Interactive playback:** Exploring Manim's OpenGL preview mode or alternative rendering backends for pause/step interactivity.

Acknowledgment

We would like to thank Dr. James Daly for his guidance, feedback, and support throughout the development of this work.

References

[1] P. J. Guo, "PythonTutor: Visualize code execution," https://pythontutor.com, 2010, accessed: Feb. 13, 2026.

[2] ——, "Online python tutor: embeddable web-based program visualization for cs education," in *Proceeding of the 44th ACM Technical Symposium on Computer Science Education*, ser. SIGCSE '13. New York, NY, USA: Association for Computing Machinery, 2013, pp. 579–584. [Online]. Available: https://doi.org/10.1145/2445196.2445368

[3] ——, "Codeopticon: Real-time, one-to-many human tutoring for computer programming," in *Proceedings of the 28th Annual ACM Symposium on User Interface Software & Technology*, ser. UIST '15. New York, NY, USA: Association for Computing Machinery, 2015, pp. 599–608. [Online]. Available: https://doi.org/10.1145/2807442.2807469

[4] C. Lattner and V. Adve, "LLVM: A compilation strategy for lifelong program analysis and transformation," in *Proceedings of the 2nd IEEE Annual CGO and ACM SIGPLAN Annual CC Symposium*, ser. CGO-CC '04. Palo Alto, CA, USA: IEEE Computer Society, 2004, pp. 60–72.

[5] LLVM Project, "LLVM language reference manual," https://llvm.org/docs/LangRef.html, 2024, accessed: 2026-02-25.

[6] J. Ellson, E. Gansner, E. Koutsofios, S. North, and G. Woodhull, "Graphviz – Open Graph Visualization Software," https://graphviz.org/, 2001, accessed: Apr. 17, 2026.

[7] G. Sanderson, "Manim – mathematical animation framework," https://github.com/3b1b/manim, 2015, accessed: Feb. 13, 2026.

[8] ——, "3blue1brown," Website, 2026, accessed: Feb. 13, 2026. [Online]. Available: https://www.3blue1brown.com/

[9] ——, "3blue1brown," Youtube Channel, March 2015, accessed: Feb. 13, 2026. [Online]. Available: https://www.youtube.com/@3blue1brown

[10] 3Blue1Brown, "Large language models explained briefly," https://www.youtube.com/watch?v=LPZh9BOjkQs, 2023, accessed: 2026-02-24.

[11] Numba Development Team, "llvmlite: A lightweight llvm python binding for writing jit compilers," https://github.com/numba/llvmlite, 2024, accessed: 2026-02-25.

[12] Manim Community Developers, "Manim community edition," https://www.manim.community/, 2024, accessed: Apr. 17, 2026.

[13] B. Houle, E. Oloyede, L. Grippo, and M. Ahmed, "LLVManim: Presentable debugging with LLVM and Manim," https://github.com/BenHoule/LLVManim, 2026, accessed: Apr. 17, 2026.

Appendix A
Projected Timeline

Week	Task
W_1 [2/08 – 2/14]	• Project proposal draft complete. • Development environment setup.
W_2 [2/15 – 2/21]	• LLVM Parsing
W_3 [2/22 – 2/28]	• Project Architecture Revision • Stack Frame Animation
W_4 [3/01 – 3/07]	• Project Repository Submission • Mohammad/Ben: Parse LLVM IR with llvmlite; extract functions, basic blocks, instructions. • Lily/Ebenezer: Build clean internal program model (CFG + metadata).
W_5 [3/08 – 3/14]	• Ebenezer/Lily: Implement CLI with automatic clang compilation. • Mohammad/Ben: Convert CFG data into initial scene graph structure.
W_6 [3/15 – 3/21]	• Ben: Finalize scene graph design (nodes, edges, animation events). • Implement layout algorithm and timing coordination.
W_7 [3/22 – 3/28]	• Ben: Generate Manim code for CFG mode. • Lily: Deliver first end-to-end rendered animation.
W_8 [3/29 – 4/04]	• Ebenezer: Extend scene graph for stack and memory modeling. • Animate alloca, load, store, call, ret.
W_9 [4/04 – 4/10]	• Mohammad: Implement trace mode with instruction highlighting and speed control.
W_{10} [4/11 – 4/17]	• All: Add advanced customization (palette, layout, YAML) and pause/step interactivity. • ALL; integrate stretch feature (lightweight GUI or IR inspection), finalize docs and demos.

TABLE III: Projected timeline for LLVManim Development.

Appendix B
Input File: double.ll

`double.ll` is a minimal C program compiled at `-O0` with full debug info (`clang -O0 -g -S -emit-llvm double.c`). It defines three functions—`@init`, `@double_1`, and `@main`—whose control flow, heap allocation, and loop structure exercise all four LLVManim animation modes.

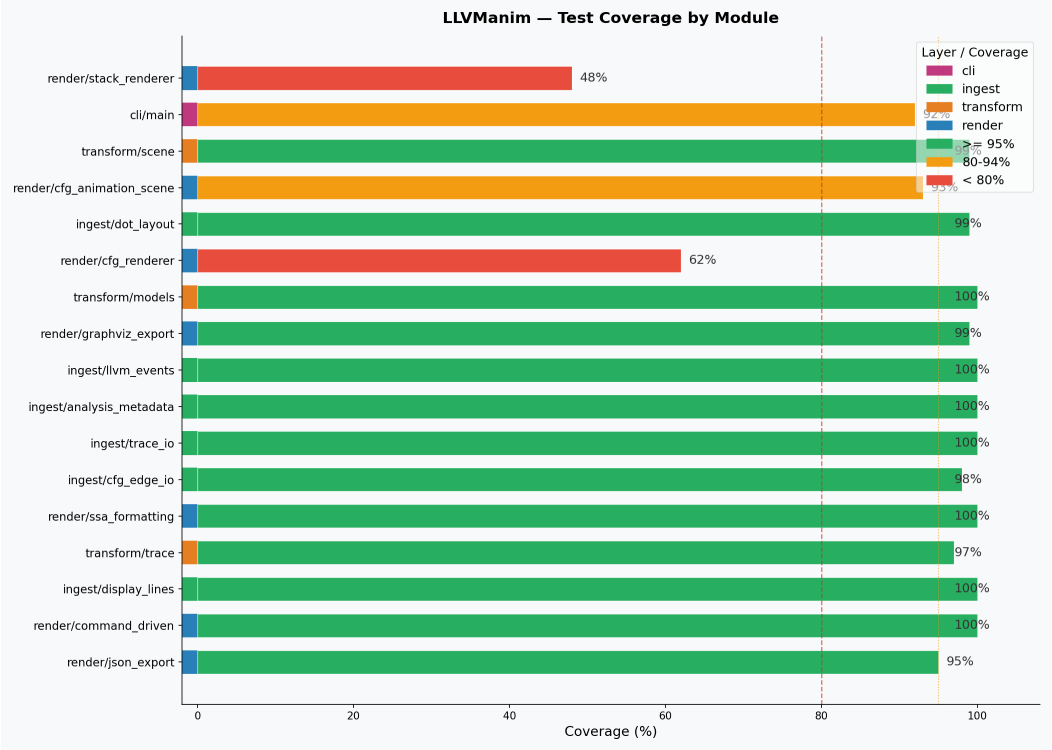
```
; ModuleID = 'double.c'
source_filename = "double.c"
target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-i128:128-f80:128-n8:16:32:64-S128"
target triple = "x86_64-pc-linux-gnu"

; Function Attrs: noinline nounwind optnone uwtable
define dso_local void @init(ptr noundef %p) #0 !dbg !13 {
entry:
  %p.addr = alloca ptr, align 8
  store ptr %p, ptr %p.addr, align 8
  call void @llvm.dbg.declare(metadata ptr %p.addr, metadata !17, metadata !DIExpression()), !dbg !18
  %0 = load ptr, ptr %p.addr, align 8, !dbg !19
  store i32 1, ptr %0, align 4, !dbg !20
  ret void, !dbg !21
}

; Function Attrs: nocallback nofree nosync nounwind speculatable willreturn memory(none)
declare void @llvm.dbg.declare(metadata, metadata, metadata) #1

; Function Attrs: noinline nounwind optnone uwtable
define dso_local i32 @double_1(ptr noundef %p) #0 !dbg !22 {
entry:
  %p.addr = alloca ptr, align 8
  %tmp = alloca i32, align 4
  store ptr %p, ptr %p.addr, align 8
  call void @llvm.dbg.declare(metadata ptr %p.addr, metadata !25, metadata !DIExpression()), !dbg !26
  call void @llvm.dbg.declare(metadata ptr %tmp, metadata !27, metadata !DIExpression()), !dbg !28
  %0 = load ptr, ptr %p.addr, align 8, !dbg !29
  %1 = load i32, ptr %0, align 4, !dbg !30
  store i32 %1, ptr %tmp, align 4, !dbg !28
  %2 = load i32, ptr %tmp, align 4, !dbg !31
```

```
  %mul = mul nsw i32 2, %2, !dbg !32
  %3 = load ptr, ptr %p.addr, align 8, !dbg !33
  store i32 %mul, ptr %3, align 4, !dbg !34
  %4 = load ptr, ptr %p.addr, align 8, !dbg !35
  %5 = load i32, ptr %4, align 4, !dbg !36
  ret i32 %5, !dbg !37
}

; Function Attrs: noinline nounwind optnone uwtable
define dso_local i32 @main() #0 !dbg !38 {
entry:
  %retval = alloca i32, align 4
  %r = alloca i32, align 4
  %p = alloca ptr, align 8
  store i32 0, ptr %retval, align 4
  call void @llvm.dbg.declare(metadata ptr %r, metadata !41, metadata !DIExpression()), !dbg !42
  store i32 0, ptr %r, align 4, !dbg !42
  call void @llvm.dbg.declare(metadata ptr %p, metadata !43, metadata !DIExpression()), !dbg !44
  %call = call noalias ptr @malloc(i64 noundef 4) #3, !dbg !45
  store ptr %call, ptr %p, align 8, !dbg !44
  %0 = load ptr, ptr %p, align 8, !dbg !46
  call void @init(ptr noundef %0), !dbg !47
  br label %while.cond, !dbg !48

while.cond:                                       ; preds = %while.body, %entry
  %1 = load ptr, ptr %p, align 8, !dbg !49
  %2 = load i32, ptr %1, align 4, !dbg !50
  %cmp = icmp slt i32 %2, 100, !dbg !51
  br i1 %cmp, label %while.body, label %while.end, !dbg !48

while.body:                                       ; preds = %while.cond
  %3 = load ptr, ptr %p, align 8, !dbg !52
  %call1 = call i32 @double_1(ptr noundef %3), !dbg !54
  store i32 %call1, ptr %r, align 4, !dbg !55
  br label %while.cond, !dbg !48, !llvm.loop !56

while.end:                                        ; preds = %while.cond
  %4 = load i32, ptr %r, align 4, !dbg !59
  ret i32 %4, !dbg !60
}

; Function Attrs: nounwind allocsize(0)
declare noalias ptr @malloc(i64 noundef) #2

attributes #0 = { noinline nounwind optnone uwtable "frame-pointer"="all" "min-legal-vector-width"="0" "no-trapping-math"="true" "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="+cmov,+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
attributes #1 = { nocallback nofree nosync nounwind speculatable willreturn memory(none) }
attributes #2 = { nounwind allocsize(0) "frame-pointer"="all" "no-trapping-math"="true" "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="+cmov,+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
attributes #3 = { nounwind allocsize(0) }

!llvm.dbg.cu = !{!0}
!llvm.module.flags = !{!5, !6, !7, !8, !9, !10, !11}
!llvm.ident = !{!12}

!0 = distinct !DICompileUnit(language: DW_LANG_C11, file: !1, producer: "Ubuntu clang version 18.1.3 (1ubuntu1)", isOptimized: false, runtimeVersion: 0, emissionKind: FullDebug, retainedTypes: !2, splitDebugInlining: false, nameTableKind: None)
!1 = !DIFile(filename: "double.c", directory: "/home/benh/LLVManim", checksumkind: CSK_MD5, checksum: "0f941b637837176c30f52c4a0313c1b1")
!2 = !{!3}
!3 = !DIDerivedType(tag: DW_TAG_pointer_type, baseType: !4, size: 64)
!4 = !DIBasicType(name: "int", size: 32, encoding: DW_ATE_signed)
!5 = !{i32 7, !"Dwarf Version", i32 5}
!6 = !{i32 2, !"Debug Info Version", i32 3}
!7 = !{i32 1, !"wchar_size", i32 4}
!8 = !{i32 8, !"PIC Level", i32 2}
!9 = !{i32 7, !"PIE Level", i32 2}
!10 = !{i32 7, !"uwtable", i32 2}
!11 = !{i32 7, !"frame-pointer", i32 2}
!12 = !{!"Ubuntu clang version 18.1.3 (1ubuntu1)"}
```

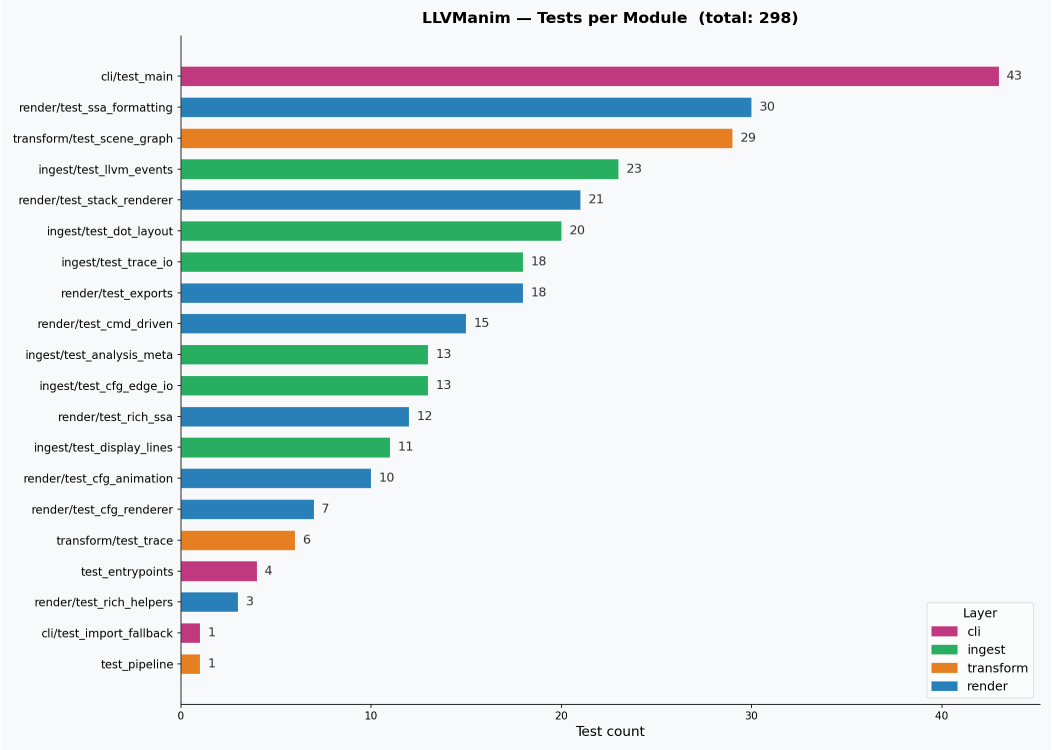
```
!13 = distinct !DISubprogram(name: "init", scope: !1, file: !1, line: 5, type: !14, scopeLine: 5, flags: DIFlagPrototyped, spFlags: DISPFlagDefinition, unit: !0, retainedNodes: !16)
!14 = !DISubroutineType(types: !15)
!15 = !{null, !3}
!16 = !{}
!17 = !DILocalVariable(name: "p", arg: 1, scope: !13, file: !1, line: 5, type: !3)
!18 = !DILocation(line: 5, column: 16, scope: !13)
!19 = !DILocation(line: 6, column: 4, scope: !13)
!20 = !DILocation(line: 6, column: 5, scope: !13)
!21 = !DILocation(line: 7, column: 1, scope: !13)
!22 = distinct !DISubprogram(name: "double_1", scope: !1, file: !1, line: 9, type: !23, scopeLine: 9, flags: DIFlagPrototyped, spFlags: DISPFlagDefinition, unit: !0, retainedNodes: !16)
!23 = !DISubroutineType(types: !24)
!24 = !{!4, !3}
!25 = !DILocalVariable(name: "p", arg: 1, scope: !22, file: !1, line: 9, type: !3)
!26 = !DILocation(line: 9, column: 19, scope: !22)
!27 = !DILocalVariable(name: "tmp", scope: !22, file: !1, line: 10, type: !4)
!28 = !DILocation(line: 10, column: 7, scope: !22)
!29 = !DILocation(line: 10, column: 14, scope: !22)
!30 = !DILocation(line: 10, column: 13, scope: !22)
!31 = !DILocation(line: 11, column: 10, scope: !22)
!32 = !DILocation(line: 11, column: 9, scope: !22)
!33 = !DILocation(line: 11, column: 4, scope: !22)
!34 = !DILocation(line: 11, column: 6, scope: !22)
!35 = !DILocation(line: 12, column: 12, scope: !22)
!36 = !DILocation(line: 12, column: 11, scope: !22)
!37 = !DILocation(line: 12, column: 3, scope: !22)
!38 = distinct !DISubprogram(name: "main", scope: !1, file: !1, line: 15, type: !39, scopeLine: 15, spFlags: DISPFlagDefinition, unit: !0, retainedNodes: !16)
!39 = !DISubroutineType(types: !40)
!40 = !{!4}
!41 = !DILocalVariable(name: "r", scope: !38, file: !1, line: 16, type: !4)
!42 = !DILocation(line: 16, column: 7, scope: !38)
!43 = !DILocalVariable(name: "p", scope: !38, file: !1, line: 17, type: !3)
!44 = !DILocation(line: 17, column: 8, scope: !38)
!45 = !DILocation(line: 17, column: 20, scope: !38)
!46 = !DILocation(line: 18, column: 8, scope: !38)
!47 = !DILocation(line: 18, column: 3, scope: !38)
!48 = !DILocation(line: 19, column: 3, scope: !38)
!49 = !DILocation(line: 19, column: 11, scope: !38)
!50 = !DILocation(line: 19, column: 10, scope: !38)
!51 = !DILocation(line: 19, column: 13, scope: !38)
!52 = !DILocation(line: 20, column: 18, scope: !53)
!53 = distinct !DILexicalBlock(scope: !38, file: !1, line: 19, column: 20)
!54 = !DILocation(line: 20, column: 9, scope: !53)
!55 = !DILocation(line: 20, column: 7, scope: !53)
!56 = distinct !{!56, !48, !57, !58}
!57 = !DILocation(line: 21, column: 3, scope: !38)
!58 = !{!"llvm.loop.mustprogress"}
!59 = !DILocation(line: 22, column: 10, scope: !38)
!60 = !DILocation(line: 22, column: 3, scope: !38)
```

Listing 1: Full LLVM IR source (`double.ll`) used for all sample animations.

Appendix C
Figures for Metrics



(a) Test coverage by module. Most modules exceed the 80% threshold; `render/stack_renderer` (48%) and `render/cfg_renderer` (62%) are the primary gaps due to Manim rendering dependencies.



(b) Test count per module (298 total). CLI, transform, and ingest modules carry the largest test loads.

Fig. 8: Test coverage and test count by module.

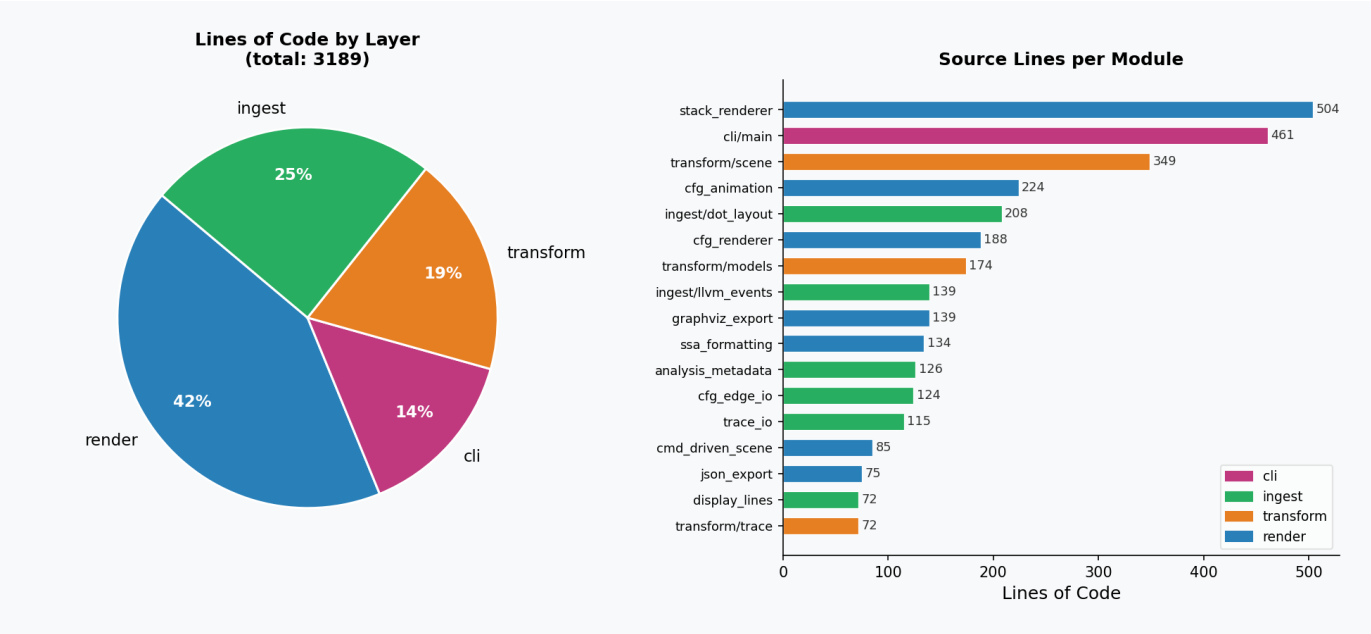


Fig. 9: Lines of code by layer (left) and per module (right). The Render layer is the largest due to the Manim animation logic in `stack_renderer` and `cfg_animation`.

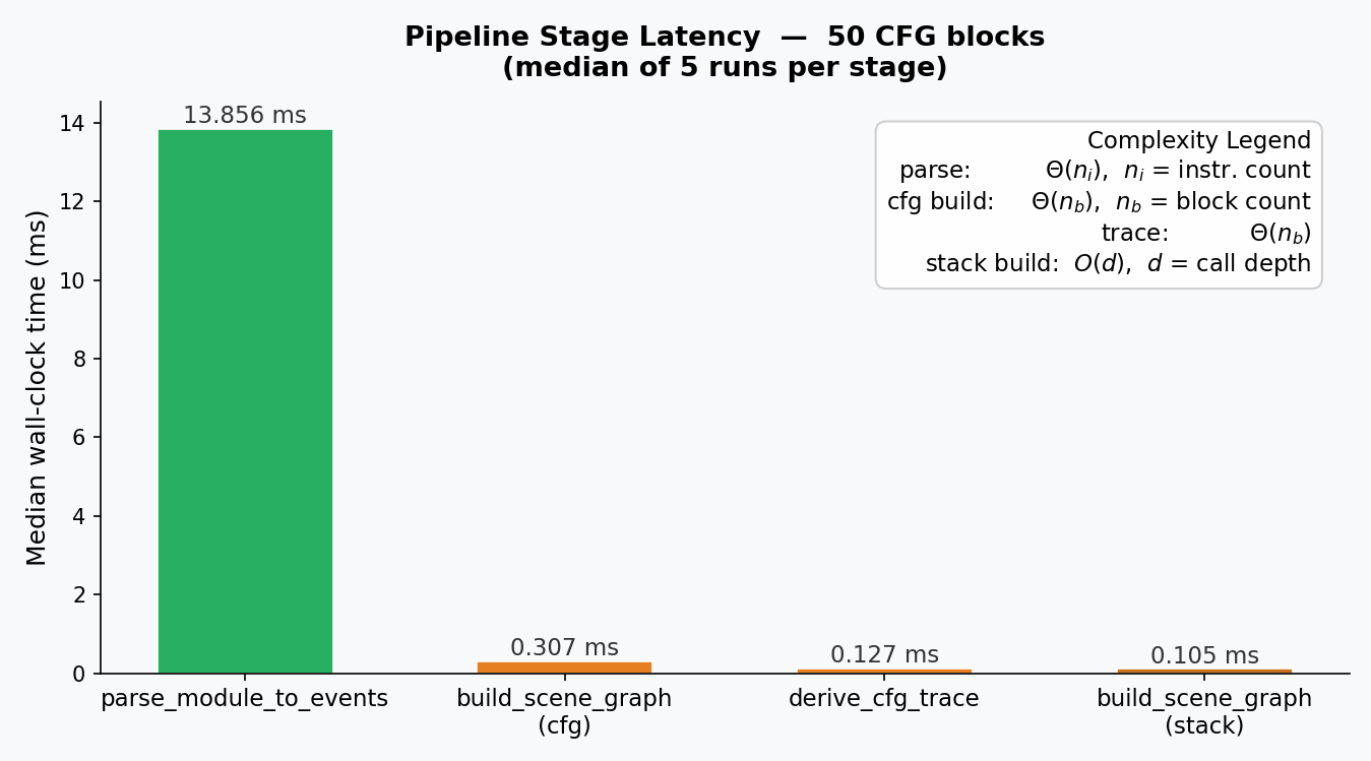


Fig. 10: Pipeline stage latency for a 50-block CFG (median of 5 runs). Parsing dominates; Transform stages are two orders of magnitude faster.

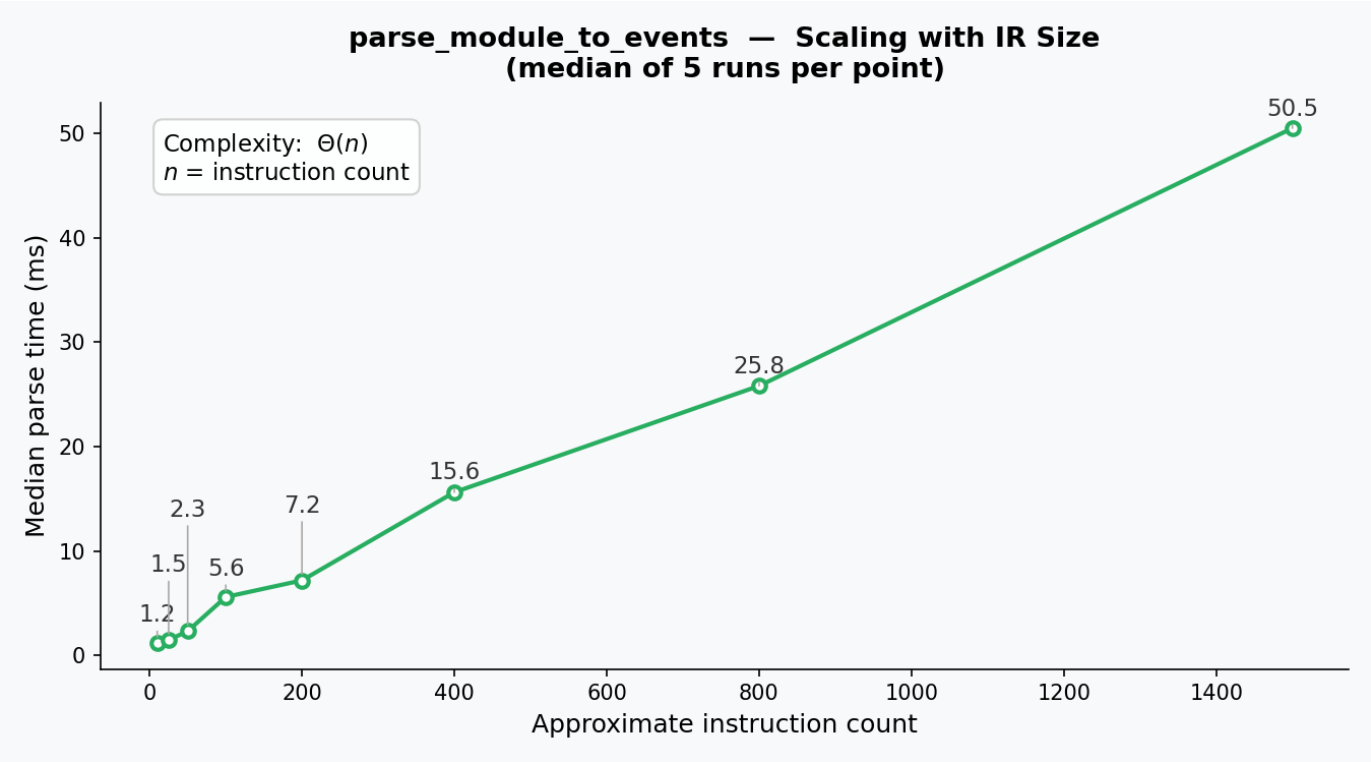


Fig. 11: `parse_module_to_events` scaling. Linear in instruction count—$\Theta(n)$.

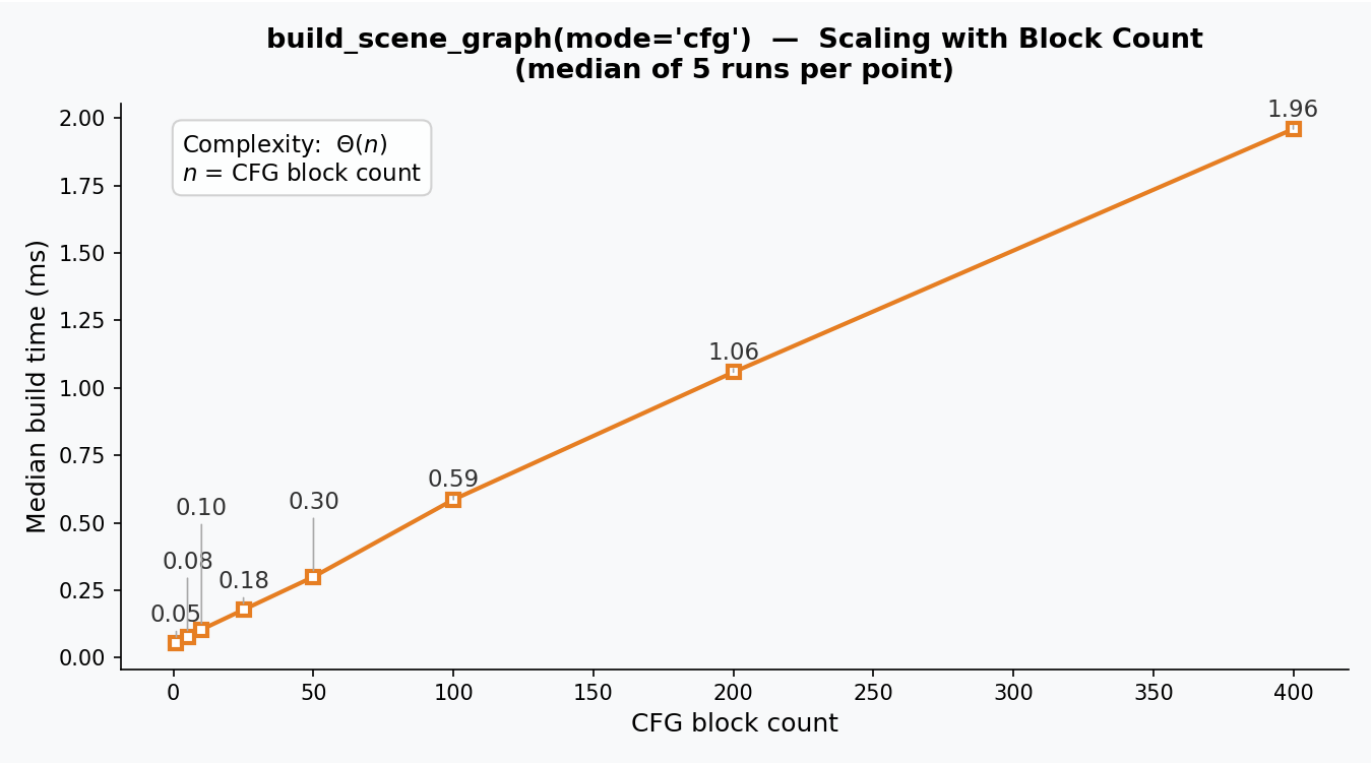


Fig. 12: CFG scene-graph build time vs. block count—$\Theta(n)$.

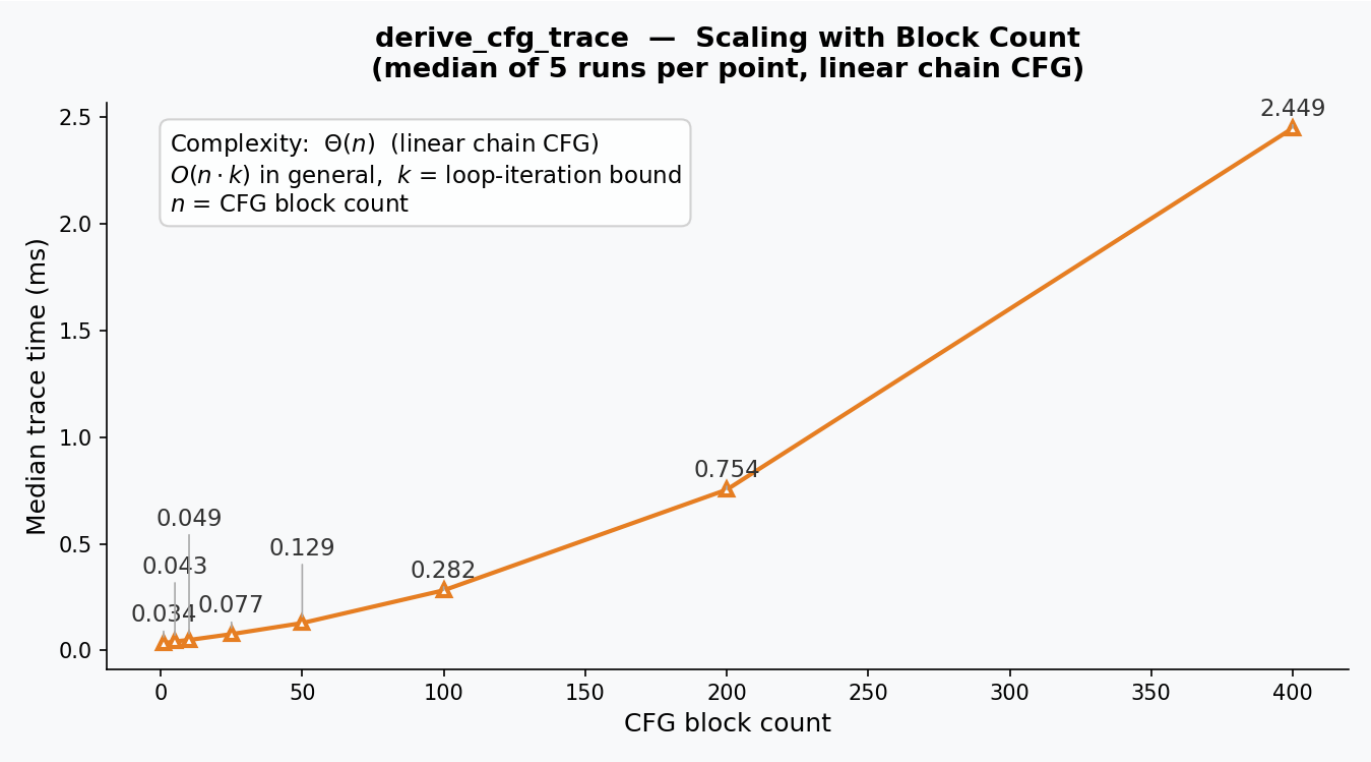


Fig. 13: CFG trace derivation scaling—$\Theta(n)$ for linear chains, $O(n \cdot k)$ with loops.

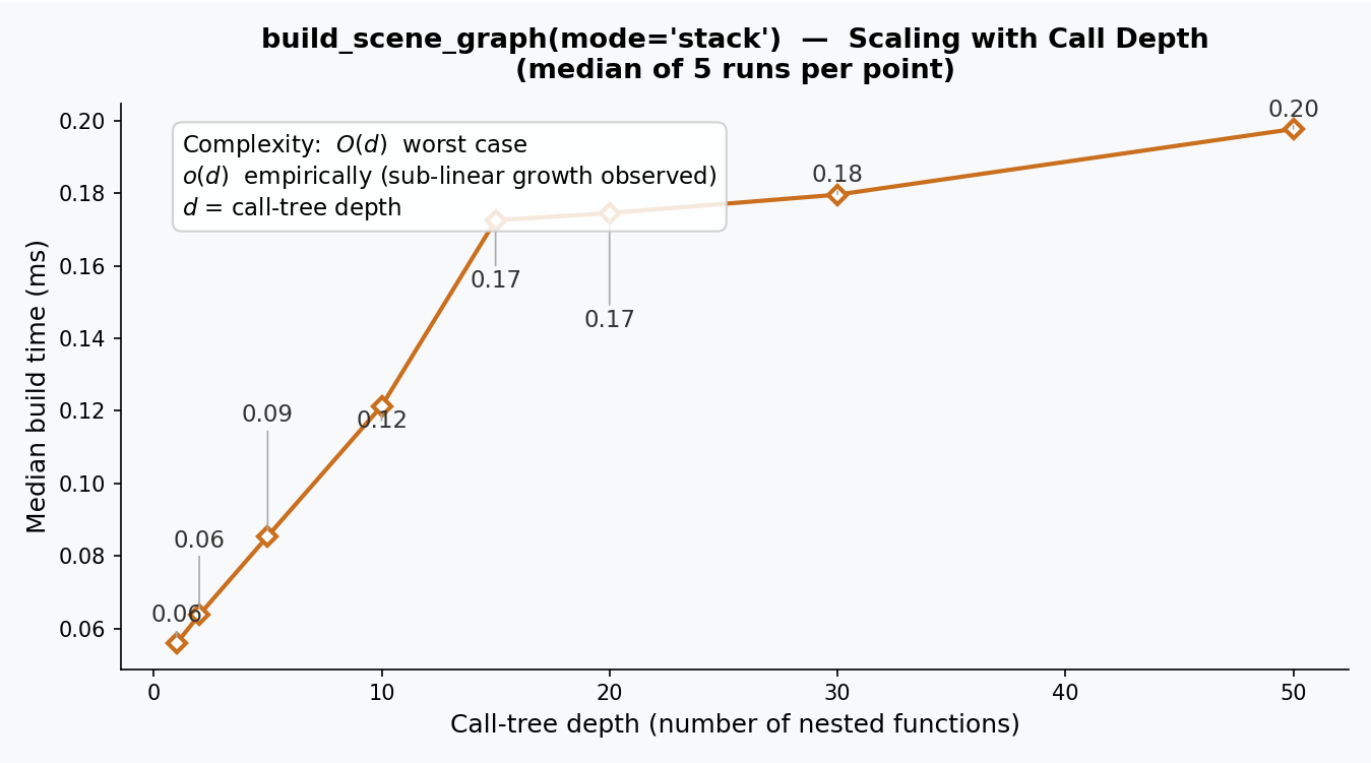


Fig. 14: Stack scene-graph build time vs. call depth—$O(d)$ worst case, sub-linear empirically.

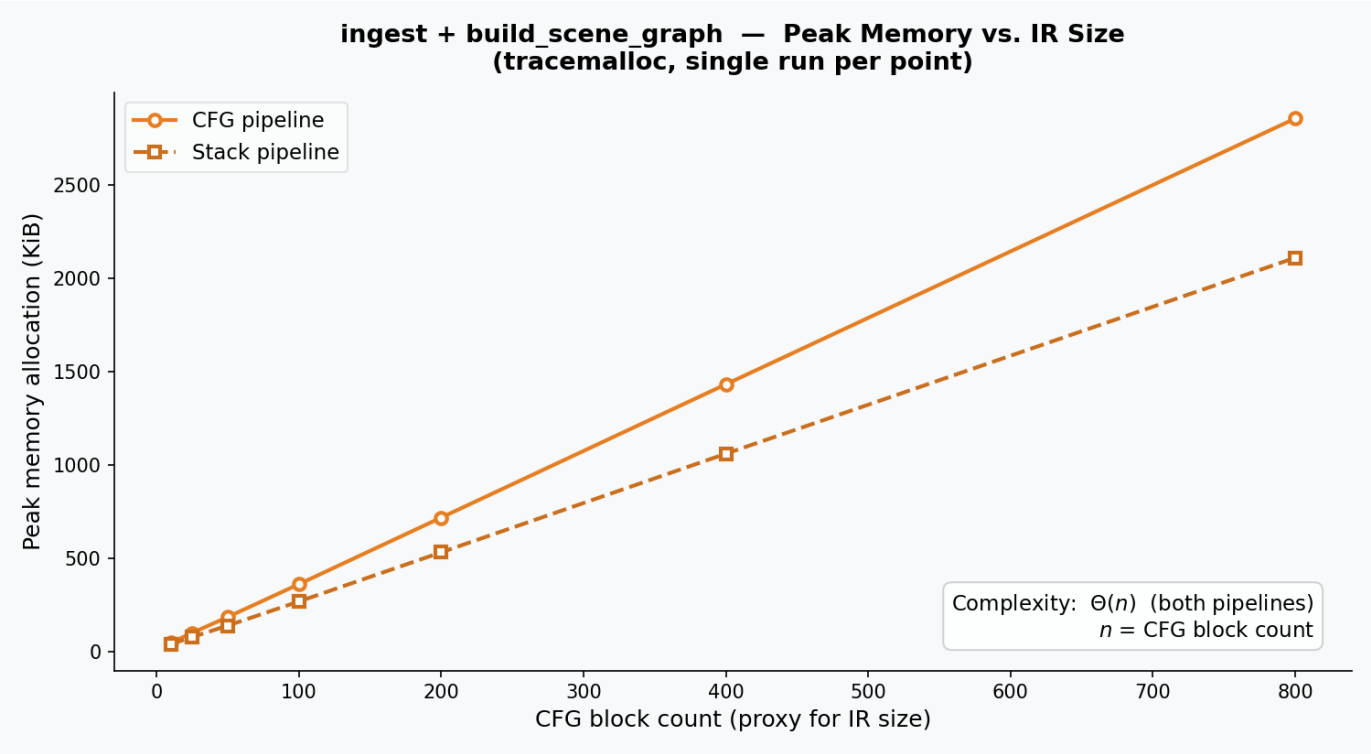


Fig. 15: Peak memory allocation vs. CFG block count. Both pipelines scale linearly—$\Theta(n)$.

APPENDIX D
FEEDBACK

[1]— *"The figures are helpful for showing how the existing tools work, but for the proposal make sure you show how you want the visualized code to work. This could be through making a mockup or setting up input for Manim to generate something that looks like what you want. You should also think about the pipeline for what data you are planning on getting from LLVM, how you are going to get it, and how you will feed it into Manim."*

[2]— *"You have figures for some existing tools. Make sure you talk about them in the proposal and what their limitations are."*

---

[3]— *"This [PythonTutor] figure is unreadable."*

[4]— *"[PythonTutor] Spelling."*

[5]— *"Which languages [are supported by PythonTutor]?"*

[6]— *"You should have a citation for where to access the [PythonTutor] tool itself and not just the paper describing its efficacy."*

[7]— *"The text in the [control-flow graph] figure is too small."*

[8]— *"[Graphviz] Linky please."*

[9]— *"Linky to Manim."*

[10]— *"citation needed [for 3Blue1Brown's winning a Streamy award]"*

[11]— *"The order of the [model] diagram is in the opposite order that they are presented in [implementation plan] text."*

[12]— *"The items in the timeline are incredibly brief and need more detail or support."*

---

[13]— *"The figures are quite shrunk down especially figure 1 and figure 2 making it extremely difficult to read the content within it, if those were made larger it would greatly improve the readability."*

[14]— *"The abstract could briefly mention the intended users explicitly, such as educators, students, or software engineers. Additionally, the abstract could briefly summarize how LLVManim achieves this goal rather than only stating the objective."*

$Response_{[14]}$— The intended users of LLVManim is a broad spectrum, including educators, students, and software engineers. We believe the abstract already addresses the context in which LLVManim is useful, and to specify the users that exist within those contexts would be redundant.

[15]— *"The introduction could more explicitly define the scope of the LLVManim system itself earlier in the section. Currently, LLVManim is only briefly mentioned at the end. Introducing LLVManim sooner would improve clarity and help frame the remainder of the document."*

[16]— *"[The PythonTutor] section could be slightly condensed. Some explanations regarding educational benefits are repeated, and the limitations relevant specifically to LLVManim's goals like their lack of customization, presentation-quality output, etc., should be emphasized more clearly. Additionally, explicitly comparing PythonTutor directly to LLVManim in one or two sentences would make the relevance stronger."*

$Response_{[16]}$— We believe the current level of detail in the PythonTutor section is appropriate for providing necessary context and motivation for LLVManim. We may provide a more explicit comparison to LLVManim in the future, but we feel the current description sufficiently highlights the limitations of PythonTutor that LLVManim aims to address.

[17]— *"[The LLVM Analysis and Transform Passes] section could be improved by separating the strengths and weaknesses into clearly defined subsections or paragraphs. Currently, strengths and limitations are interwoven, which makes it slightly harder to skim. Additionally, some technical explanations like the IR syntax examples could be shortened slightly, as the focus of the proposal is visualization rather than compiler internals."*

$Response_{[17]}$— We believe the current structure of the LLVM Analysis and Transform Passes section effectively conveys the necessary information without overcomplicating the overall document structure. It follows the same format as the other sections describing existing tools, and we feel it is consistent with the overall style of the proposal.

[18]— *"[The Manim] section could more explicitly explain how Manim integrates into the LLVManim system pipeline. Additionally, some historical background about 3Blue1Brown could be slightly condensed to maintain focus on the project itself."*

$Response_{[18]}$— We reduced the historical background about 3Blue1Brown. However, these sections are focused on providing context for the tools we are using, and we feel that the current level of detail is appropriate.

[19]— *"Some proof-of-concept metrics (such as $\geq 95\%$ trace mapping and $\geq 80\%$ usability success rate) could briefly explain how these metrics will be measured or evaluated. This would strengthen the feasibility and evaluation clarity. Additionally, the distinction between core features and major features could be clarified slightly, as both appear required for the full system."*

[20]— *"The architecture diagram should be explicitly referenced and described more clearly in the text. Explaining what each arrow and component represents would improve clarity. Additionally, some implementation details (such as exact timing offsets and layout algorithms) may be overly detailed for a proposal and could be slightly condensed."*

[21]— *"The output format could be explained more clearly, particularly how it connects to the Transformation layer."*

[22]— *"The scene graph structure could benefit from a brief visual example or mock representation to clarify how data flows through the system. Additionally, the modified Sugiyama layout algorithm is mentioned but not briefly explained, which may confuse readers unfamiliar with graph layout techniques."*

[23]— *"The relationship between scene graph generation and Manim output could be explained more clearly in terms of data flow. Additionally, it could briefly explain expected output formats and user workflows."*

[24]— *"[The User Interface] section could briefly explain error handling and limitations more clearly, such as unsupported language features or missing debug metadata. Additionally, this section could clarify whether the CLI is intended for developers, educators, or both."*

[25]— *"Testing and validation phases could be more clearly defined, particularly usability testing and feature validation. Additionally, stretch feature implementation is only briefly mentioned and could be explicitly identified in the timeline."*

$Response_{[25]}$— We have included testing and validation as part of the overall implementation plan, but we have not explicitly defined separate phases for these activities. We will be continuously testing and validating our implementation throughout the development process, and we will prioritize core features before stretch features to ensure we have a MVP by the end of the semester.

[26]— *"The feedback section [...] could be improved by explicitly explaining how each feedback item will be addressed moving forward rather than only stating that it remains unaddressed. Additionally, creating mockups or example animation output would significantly strengthen the proposal and demonstrate feasibility."*

$Response_{[26]}$— We have addressed feedback in the manner prescribed by the project guidelines, which is to either address it or explain why it remains unaddressed.

[27]— *"Ensure consistent formatting across all references and verify citation formatting matches IEEE or required course format exactly."*

$Response_{[27]}$— We have reviewed our references and believe they are formatted correctly according to the IEEE style. If there are any specific formatting issues, we would appreciate further clarification so we can address them appropriately.

---

[28]— *"It's unnecessary to explain how Julia works. It distracts from the main focus of the paper."*

$Response_{[28]}$— We do not explain how Julia works in the proposal. We have reworded the relevant section for clarity.

[29]— *"You don't have to include all these accomplishments. We already get the idea that [3Blue1Brown] is popular and well respected from earlier."*

[30]— *"Including pictures of Manim diagrams here would illustrate your point better."*

[31]— *"Your goals seem overly ambitious for just a one semester student project."*

$Response_{[31]}$— We acknowledge that our goals are ambitious, but we believe they are achievable with careful planning and prioritization. We also believe that our approach to the project is more relevant to the goals of the class than the final output, and we are committed to making significant progress towards our objectives regardless of the final scope of the project.

[32]— *"It's unrealistic to expect work to be done over spring break."*

$Response_{[32]}$— We appreciate the concern, but our teammembers are aware of the timeline and have planned accordingly.

[33]— *"You should talk more about the potential risks. It's not really mentioned at all."*

[34]— *"The motivation section could be strengthened. While the proposal explains that existing tools are either too technical or not presentation-friendly, it does not fully explain why this problem is important at a broader level. It would help to more clearly identify who specifically benefits from this tool and why current alternatives are insufficient. The proposal currently reads as though the improvement is mostly aesthetic, so emphasizing the educational or practical impact would make the motivation more compelling."*

$Response_{[34]}$— We believe the motivation section sufficiently explains the importance of the problem and the intended beneficiaries of LLVManim. We feel that the educational and practical impact of LLVManim is clear from the context provided, and, frankly, the improvement is primarily aesthetic.

[35]— *"[some objectives] could be more measurable. For example, the proposal mentions performance and trace mapping goals, but it would be helpful to specify what constitutes success more concretely. Adding clearer success criteria would make it easier to evaluate whether the project meets its goals."*

[36]— *"The [architecture] scope could be clarified further. It would be helpful to specify what subset of LLVM IR will be fully supported and how unsupported constructs will be handled. Addressing potential edge cases would make the approach feel more complete and realistic."*

[37]— *"The proposal would benefit from a clearer description of final deliverables. While features are described, it is not entirely clear what concrete artifacts will be submitted at the end of the project. Explicitly listing tangible deliverables would strengthen this section."*

[38]— *"The timeline is present but somewhat broad in places. Some weekly goals are described generally, which makes it difficult to determine what measurable progress will be achieved each week. Making each milestone more concrete and demonstrable would better align with the rubric's expectations."*

[39]— *"The proposal mentions evaluation goals such as trace mapping percentages and usability testing, which is good. However, the evaluation methodology could be described in more detail. For example, specifying the number of participants in the pilot study, the types of tasks they will perform, and how results will be measured would make the evaluation plan stronger and more rigorous."*

[40]— *"A clearly labeled risks section would improve the proposal. While potential challenges are implied, explicitly identifying technical and timeline risks*

and describing mitigation strategies would demonstrate thoughtful project planning."

---

[41]— *"I feel [the PythonTutor section] should include the supported languages of PythonTutor, since the usefulness of the tool depends heavily on what languages are supported. It is more useful if it supports languages that are commonly used for education vs if it supports languages that are more obscure."*

[42]— *"What are DOT files? does this mean unix dot files, or does this stand for something?"*

[43]— *"What does verbosity mean in this application and how is it different from animation speed. Does the creator of the animation include an audio track, or is it the speed that words appear? If the latter, how and why are words appearing?"*

$Response_{[43]}$— Verbosity in this context refers to the amount of information displayed in the animation, such as whether variable values are shown or only control flow is visualized. It is different from animation speed, which controls how quickly the animation plays. This is a common feature in debugging tools, and we feel it is reasonable to expect readers to understand this distinction.

[44]— *"Go more in depth about what the study entails. Will you be comparing against other similar tools, or just compared to only reading the code? Will it be between subjects or within subjects? Are you only going to collect data about accuracy or will you also collect data about subjective user preference? Also this study does not seem like it connects well to the stated goal of this project. You are discussing making a tool that will help educational scenarios, but are only evaluating it for use in debugging. I would add a component focusing on user experience generating the images, or evaluating which ones help people to better understand the purpose of code. There are also no sample graphs or tables for the data that is to be collected."*

[45]— *"Why separate core features from MVP features when core features are defined as "required to have a usable product at all", which is essentially the MVP?"*

[46]— *"How are animations utilized. Is it moving arrows to demonstrate code flow? Will sections of the code be moved on screen only when relevant, and removed afterward. For someone unfamiliar with Manim this is unclear."*

$Response_{[46]}$— A major portion of this project will be dedicated to exploring precisely this question. While we have some initial mockups and ideas for how the animations will work, these likely will not be finalized until towards the end of the project.

[47]— *"While I understand that this document is written with the assumption that the reader has a computer science bachelors degree, I think it would help to use the long form of this abbreviation at least once, as was done with LLVM IR."*

[48]— *"Why make your own "JSON-like" format instead of just using standard JSON. If it is for data streaming purposes, perhaps consider JSON Lines."*

[49]— *"No discussion of risks or costs."*

[50]— *"Timeline does not say when the experiment will be performed."*

[51]— *"Missing time spent on project."*

[52]— *"The lack of Manim examples or proposed examples for this project makes it difficult for someone unfamiliar with that tool to understand what this tool will produce."*

---

[53]— *Is [PythonTutor] the only such tool that you were able to find?*

[54]—*using*

[55]—*This figure is borderline unreadable.*

[56]— *I'm not sure how valuable this [3Blue1Brown] paragraph is. It feels like it should be trimmed down and come before the previous one.*

[57]— *How is this different than the IR Instruction Coverage core feature?*

[58]— *None of the stuff in the caption is represented in the [architecture] figure.*

[59]— *The [architecture] figure doesn't really match the text in section IV-b either.*

[60]— *This [mock output] figure is not readable.*

[61]— *You also need the input (both the source file to be parsed and the command that produces the output)*

[62]—*[LLVMLite is] also not in your diagram.*

HOURS SPENT: ~200 Hours